



中山大學

SUN YAT-SEN UNIVERSITY

数据安全和隐私保护

数据安全和隐私保护第一次实验

姓 名 _____ 姓 名 _____

学 号 _____ 学 号 _____

学 院 _____ 网络空间安全学院

专 业 _____ 网络空间安全

2026 年 9 月 10 日

目录

1 实验要求	1
2 实验原理	1
2.1 位图与矢量图	1
2.2 Git 的三个区域	1
2.3 Codex 与 CC Switch	1
3 实验设置	2
4 实验过程	2
4.1 LaTeX 图片排版	2
4.2 Git 与 GitHub	4
4.2.1 配置核对	4
4.2.2 创建仓库并克隆	4
4.2.3 第一次提交	4
4.2.4 上传到远程仓库	5
4.2.5 第二次提交	6
4.2.6 同步远程更新	7
4.3 Codex 辅助编程	7
4.3.1 版本与配置核对	7
4.3.2 只读阅读项目	8
4.3.3 生成程序与测试	8
4.4 CC Switch 供应商配置与切换	10
5 实验总结	11

1 实验要求

本次实验围绕科研过程中的基础工具展开，包含三个部分。

1. LaTeX 图片排版：掌握 `\includegraphics` 的使用方法，能够设置图片尺寸，为图片配置图编号、图标题并在正文中引用；优先使用 PDF、SVG、EPS 等矢量格式，避免使用低分辨率截图作为结果图。报告中至少插入一张实验结果图。
2. Git 与 GitHub：完成 Git 的安装与配置，完成仓库创建、克隆、提交、上传与更新。
3. Codex 与 CC Switch：完成 Codex 的安装与基本配置，安装 CC Switch 并完成供应商配置与切换，使用 Codex 完成至少一次任务。

按实验说明，Git、Codex、CC Switch 已在此前安装并配置完成的情况下可跳过安装步骤。本报告对应的三项工具均已在实验前完成安装与配置，因此第 4 节记录的是配置状态核对与实际操作过程。报告中涉及 API Key、Endpoint 凭据的位置均已脱敏。

2 实验原理

2.1 位图与矢量图

位图由像素矩阵组成，放大后受采样密度限制，边缘会出现锯齿或模糊。矢量图记录曲线、文字和几何对象的描述，渲染时按输出分辨率重新光栅化，缩放后仍保持清晰。折线图、柱状图、网络结构图等由绘图程序生成的图形，可直接导出为 PDF 或 SVG，避免经过截图带来的清晰度损失。

LaTeX 通过 `graphicx` 宏包将图片按版面尺寸排入文档。`figure` 环境提供浮动定位，`\caption` 生成图题与图编号，`\label` 与 `\ref` 建立正文引用关系。

2.2 Git 的三个区域

Git 是分布式版本控制系统。工作区保存正在编辑的文件，暂存区保存准备提交的变化，`commit` 将暂存内容记录到本地仓库，`push` 将本地提交上传到远程仓库，`pull` 取得远程提交并合并到当前分支。`status`、`diff`、`log` 用于在提交前后核对状态、内容差异和历史记录。数据流向如图 1 所示。

2.3 Codex 与 CC Switch

Codex 在用户授权范围内读取项目、修改文件并运行本地命令，根据自然语言需求和仓库上下文制定并执行操作。生成结果可能存在错误、遗漏或不安全修改，因此流程中保留权限确认、命令审阅、程序运行、测试和 Git 差异检查等核对环节。

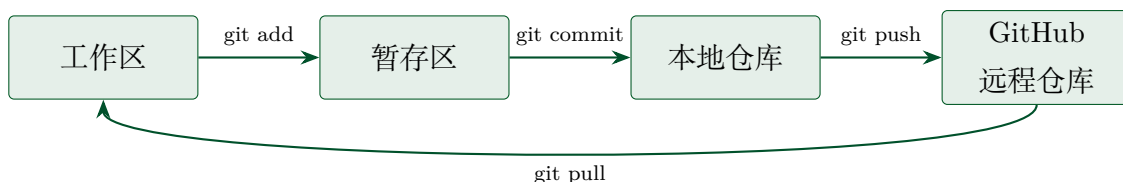


图 1: Git 工作区、暂存区、本地仓库与远程仓库之间的数据流向

CC Switch 将不同供应商的端点、模型和认证配置保存为可切换条目。切换时更新 Codex 的配置文件 `~/.codex/config.toml`，命令行工具重启后生效。使用第三方供应商时，密钥保护、端口暴露和服务条款均需关注。

3 实验设置

实验在本地 macOS 主机上完成工具操作，LaTeX 报告在远程 Linux 服务器上使用 XeLaTeX 编译。软件版本见表 1。

表 1: 实验环境

项目	版本或配置
操作系统（工具操作）	macOS, Darwin 27.0.0, arm64
操作系统（报告编译）	Linux 服务器，通过 SSH 访问
Git	2.50.1 (Apple Git-155)
Python	3.9, matplotlib 3.9.4
Codex CLI	0.153.4
CC Switch	3.14.1 (Homebrew Cask 安装)
LaTeX	TeX Live, XeLaTeX 引擎
报告模板	SYSUReport.cls

实验仓库地址为 <https://github.com/Candlest/research-tools-week1>，目录结构为 `code/`、`result/`、`report/`、`figures/` 四类。

4 实验过程

4.1 LaTeX 图片排版

实验结果图由 `code/plot_word_freq.py` 生成。该程序读取 `result/word_freq.csv`，使用 matplotlib 的 Agg 后端绘制横向柱状图，并直接导出为 PDF 矢量文件，未经过截图环节。关键设置为 `pdf.fonttype=42`，使文字以可缩放字形嵌入。

```
$ python3 code/plot_word_freq.py
wrote figures/word_freq.pdf

$ file figures/word_freq.pdf
figures/word_freq.pdf: PDF document, version 1.4, 1 pages
```

Listing 1: 生成矢量结果图

在导言区加载 `graphicx` 宏包后，按下列方式插入图片。`width` 以 `\textwidth` 为基准设置尺寸，`\caption` 与 `\label` 分别提供图题和引用标签。

```
1 \usepackage{graphicx}
2 \begin{figure}[htbp]
3   \centering
4   \includegraphics[width=0.80\textwidth]{figures/word_freq.pdf}
5   \caption{sample.txt 词频统计结果（前 10 个词）}
6   \label{fig:wordfreq}
7 \end{figure}
```

Listing 2: 图片插入代码

插入结果如图 2 所示。图中横轴为出现次数，纵轴为词。将 PDF 页面放大后，坐标轴标签、刻度数字和数值标注保持清晰。

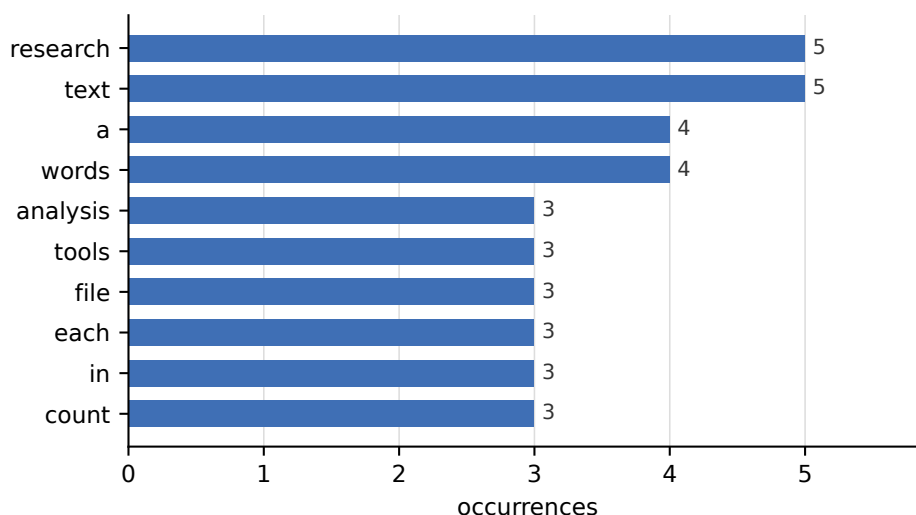


图 2: sample.txt 词频统计结果（前 10 个词）

4.2 Git 与 GitHub

4.2.1 配置核对

Git 已在实验前安装，提交身份已配置。核对结果如下，配置文件中不含密码、令牌或代理设置。

```
$ git --version
git version 2.50.1 (Apple Git-155)

$ git config --global --list
user.email=candlest1112@gmail.com
user.name=Candlest
http.version=HTTP/1.1
alias.dotfiles=!git --git-dir=$HOME/.dotfiles --work-tree=$HOME
```

Listing 3: Git 版本与用户级配置

4.2.2 创建仓库并克隆

在 GitHub 创建公开仓库 `research-tools-week1` 并初始化 README，随后克隆到本地并建立 `code/`、`result/`、`report/`、`figures/` 四个目录。空目录不会被 Git 记录，因此在 `result/` 和 `report/` 中各放置一个说明文件。

```
$ gh repo create research-tools-week1 --public --add-readme
https://github.com/Candlest/research-tools-week1

$ git clone https://github.com/Candlest/research-tools-week1.git
Cloning into 'research-tools-week1'...

$ cd research-tools-week1
```

Listing 4: 创建并克隆仓库

4.2.3 第一次提交

提交前依次执行 `status` 和 `diff`，确认变化范围与内容，再明确列出待暂存文件，未使用 `git add .`。

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
```

```
        modified:   README.md

Untracked files:
  code/
  report/
  result/

$ git diff
diff --git a/README.md b/README.md
index 16bd6b3..4e73d8d 100644
--- a/README.md
+++ b/README.md
@@ -1,2 +1,19 @@
+ # research-tools-week1
-...
+...

$ git add README.md code/demo.py result/README.md report/README.md

$ git diff --staged --stat
README.md      | 19 ++++++++
code/demo.py   | 21 ++++++++
report/README.md | 1 +
result/README.md | 1 +
4 files changed, 41 insertions(+), 1 deletion(-)

$ git commit -m "add project layout and first runnable demo program"
[main 092cb4d] add project layout and first runnable demo program
4 files changed, 41 insertions(+), 1 deletion(-)
```

Listing 5: 检查修改并创建第一次提交

4.2.4 上传到远程仓库

首次向 main 分支上传时使用 -u 建立上游关系。

```
$ git branch --show-current
main

$ git push -u origin main
To https://github.com/Candlest/research-tools-week1.git
```

```
03db75d..092cb4d  main -> main
branch 'main' set up to track 'origin/main'.

$ git log --oneline --graph --decorate -n 10
* 092cb4d (HEAD -> main, origin/main, origin/HEAD) add project layout
  and first runnable demo program
* 03db75d Initial commit
```

Listing 6: 建立上游关系并上传

4.2.5 第二次提交

第二次提交加入 Codex 生成的词频统计程序、单元测试、绘图程序、样例输入、统计结果 CSV 和结果图 PDF。

```
$ git add code/text_stats.py code/test_text_stats.py code/
  plot_word_freq.py \
    result/sample.txt result/word_freq.csv figures/word_freq.pdf

$ git diff --staged --stat
code/plot_word_freq.py | 44 +++++
code/test_text_stats.py | 61 +++++
code/text_stats.py      | 35 +++++
figures/word_freq.pdf   | Bin 0 -> 10824 bytes
result/sample.txt       | 16 +++++
result/word_freq.csv    | 9 +++++
6 files changed, 165 insertions(+)

$ git commit -m "add word frequency program, unit tests and result
  figure"
[main d6f5db7] add word frequency program, unit tests and result
  figure
6 files changed, 165 insertions(+)

$ git push
To https://github.com/Candlest/research-tools-week1.git
092cb4d..d6f5db7  main -> main
```

Listing 7: 第二次提交并上传

4.2.6 同步远程更新

在 GitHub 端修改 README 中的一行文字并提交，随后在本地确认工作区干净并执行 `git pull`。本次为快进合并，未出现冲突。

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

$ git pull
From https://github.com/Candlest/research-tools-week1
   d6f5db7..457f9bd  main      -> origin/main
Updating d6f5db7..457f9bd
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git log --oneline --graph --decorate -n 10
* 457f9bd (HEAD -> main, origin/main, origin/HEAD) edit README line
  on GitHub web
* d6f5db7 add word frequency program, unit tests and result figure
* 092cb4d add project layout and first runnable demo program
* 03db75d Initial commit
```

Listing 8: 拉取远程更新

历史记录中包含两次本地提交和一次远程更新，与实验要求一致。

4.3 Codex 辅助编程

4.3.1 版本与配置核对

Codex CLI 已在实验前安装。当前通过 API Key 方式使用第三方供应商，因此 `codex login status` 显示未使用 ChatGPT 账号登录。

```
$ codex --version
codex-cli 0.153.4

$ codex login status
Not logged in
```

Listing 9: Codex 版本与登录状态

用户级配置位于 `~/.codex/config.toml`，关键字段如下，其中 API Key 由 CC Switch 写入 `auth.json` 并已脱敏。

```

1 model_provider = "custom"
2 model = "gpt-6-astra"
3 model_reasoning_effort = "low"
4
5 [model_providers.custom]
6 name = "<REDACTED>"
7 base_url = "<REDACTED>"
8 wire_api = "responses"

```

Listing 10: config.toml 关键字段（脱敏）

4.3.2 只读阅读项目

第一步限制 Codex 为只读沙盒，要求其阅读项目并说明结构，不修改文件。

```

$ codex exec --sandbox read-only \
    "read the project, describe the layout, entry point and how to
    run it, do not modify files"

```

Listing 11: 要求 Codex 只读阅读项目

Codex 输出的目录结构与入口说明如下，与仓库实际内容一致。它同时指出 README 中提到的 `code/text_stats.py` 与 `result/sample.txt` 当时尚不存在，这一判断正确。

```

1 research-tools-week1/
2 |-- README.md           # project description and run commands
3 |-- code/
4 |   |-- demo.py         # character and line counting
5 |-- result/README.md    # results and figures
6 |-- report/README.md    # LaTeX report
7 |-- figures/            # vector figures for the report
8
9 Entry point: main() in code/demo.py. Standard library only, Python
  3.9+.

```

Listing 12: Codex 阅读结果

4.3.3 生成程序与测试

第二步在 `workspace-write` 沙盒下要求 Codex 编写词频统计程序、样例输入和单元测试。任务描述明确了切分规则、排序方式、参数形式和输出文件。

```
$ codex exec --sandbox workspace-write \  
  "add code/text_stats.py: read a text file, count English word  
    frequency  
    (lowercase, split on non-letters, drop empty tokens), print the  
      top N words  
    in descending order with --top (default 10), and write result/  
      word_freq.csv  
    with header word,count. Also add result/sample.txt (English, 15+  
      lines) and  
    code/test_text_stats.py with at least 3 unittest cases. Run the  
      tests.  
    Standard library only."
```

Listing 13: 要求 Codex 完成编程任务

Codex 新增了三个文件并自行运行了测试。退出 Codex 后使用 Git 独立核对修改范围，未发现预期外的文件变动。

```
$ git status --short  
?? code/test_text_stats.py  
?? code/text_stats.py  
?? result/sample.txt  
?? result/word_freq.csv  
  
$ python3 code/text_stats.py result/sample.txt --top 10  
research          5  
text              5  
a                 4  
words             4  
analysis          3  
tools             3  
file              3  
each              3  
in                3  
count             3  
  
$ python3 -B -m unittest discover -s code -p 'test_*.py' -v  
test_cli_default_and_custom_top_with_csv ... ok  
test_descending_frequency_and_top_limit ... ok  
test_empty_tokens_are_filtered ... ok  
test_lowercase_and_nonletter_separators ... ok
```

```
Ran 4 tests in 0.089s
```

```
OK
```

Listing 14: 核对修改并运行程序与测试

4 个测试全部通过，覆盖大小写转换与非字母切分、空串过滤、降序排序与数量限制、命令行参数与 CSV 内容。生成的 `result/word_freq.csv` 作为图 2 的数据来源。核对无误后执行第二次提交并上传。

4.4 CC Switch 供应商配置与切换

CC Switch 3.14.1 已通过 Homebrew Cask 安装，供应商条目保存在 `~/.cc-switch/cc-switch.db` 中，按 Codex、Claude Code、Gemini 等工具分类。Codex 分类下当前启用的条目名称为 `pro20x`，该条目携带的配置片段如下（API Key 与 Endpoint 已脱敏）。

```
1 {
2   "auth": { "OPENAI_API_KEY": "<REDACTED>" },
3   "config": "model_provider = \"custom\"
4             model = \"gpt-5.5\"
5             model_reasoning_effort = \"medium\"
6
7             [model_providers.custom]
8             name = \"OpenAI\"
9             base_url = \"<REDACTED>\"
10            wire_api = \"responses\"
11            requires_openai_auth = false
12 }
```

Listing 15: CC Switch 中启用条目携带的 Codex 配置（脱敏）

配置与切换的操作步骤为：在 CC Switch 的 Codex 页面点击添加供应商；选择内置预设，或选择自定义后按供应商文档填写名称、API Key、Endpoint URL 和模型，Codex 端点通常以 `/v1` 结尾；保存后点击供应商卡片上的启用；关闭并重新打开 Codex 或终端，使新的供应商和模型列表生效；在会话中用 `/status` 核对当前模型与权限。

切换的生效机制为：CC Switch 将所选条目的 `config` 字段写入 `~/.codex/config.toml`，将 `auth` 字段写入 `~/.codex/auth.json`。命令行工具在启动时读取配置文件，因此切换后需要重启 Codex CLI 或 Codex App。前述两次 `codex exec` 调用即在当前启用的供应商下完成，返回结果正常，可作为供应商配置生效的验证。

5 实验总结

本次实验完成了三部分内容。LaTeX 部分建立了图片规范：结果图由绘图程序直接导出为 PDF 矢量文件，通过 `\includegraphics` 按 `\textwidth` 比例设置尺寸，并配置图题、标签与正文引用。Git 部分完成了仓库创建、克隆、两次本地提交、上传和一次远程更新的完整流程，提交前均通过 `status` 与 `diff` 核对变化范围。Codex 部分先以只读沙盒确认项目理解正确，再在 `workspace-write` 沙盒下完成编程任务，退出后用 Git 独立核对修改并运行测试。CC Switch 部分核对了供应商条目的存储位置和切换生效机制。

实验过程中形成的几点认识如下。第一，先执行只读任务再执行写入任务，可以在授予写权限前确认工具对项目的理解是否正确。第二，Codex 生成的修改需要通过 `git status` 和 `git diff` 独立核对，工具自身报告的修改范围不作为唯一依据。第三，矢量图应由绘图程序在数据阶段直接导出，重新截图会使前一步的清晰度损失无法恢复。第四，API Key 出现在 CC Switch 界面、`auth.json` 和终端输出中，报告与仓库中的对应位置需要逐一脱敏。